UNIVERSAL DISPLAY DRIVER

LED, LCD & OLED Based Unified Arduino Firmware System

A PROJECT REPORT

Submitted By:

Priyanshu Gupta (2023344470)

Paarth Srivastava (2023373090)

Preeti Pal (2023310314)

SHARDA UNIVERSITY

Academic Year 2025-2026

CERTIFICATE

This is to certify that the project entitled "Universal Display Driver: LED, LCD & OLED Unified Arduino Firmware System" is a bonafide work carried out by Priyanshu Gupta, Preeti Pal, Paarth Srivastava in partial fulfillment of the requirements for the a assignment submission of reverse learning during the academic year 2025–2026.

Project Guide: Dr. Zatin Gupta

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide for continuous guidance, valuable suggestions, and technical support throughout the development of this project. I also thank the faculty members of the Department of Computer Science and Engineering for providing laboratory facilities and encouragement.

I am grateful to my classmates and friends who helped in testing the hardware setup and troubleshooting wiring and software issues.

1. Abstract

This project presents the design and implementation of a Universal Display Driver that supports LED (7-Segment), 16x2 LCD, and 0.96” OLED displays using a single Arduino firmware architecture. In embedded system development, different display technologies require different driver libraries, communication protocols, initialization sequences, and memory handling mechanisms. As a result, replacing one display with another often requires rewriting a significant portion of the application code.

The proposed system solves this issue by introducing a modular and layered firmware structure in which the application layer remains independent of the hardware layer. A unified display interface is designed so that the same display function can operate across multiple display types without modifying the core logic. The Arduino Uno microcontroller serves as the central controller responsible for communication management, display initialization, and output handling.

Experimental testing confirms that the system performs reliably with all three display modules. The approach reduces firmware redundancy, improves maintainability, simplifies debugging, and enhances flexibility for prototyping, IoT systems, robotics interfaces, and educational laboratory experiments.

2. Introduction

Embedded systems form the foundation of modern electronic devices. From digital clocks and smart meters to robotics and IoT-based systems, microcontrollers are widely used to control sensors, actuators, and user interfaces. Among these components, display modules play a crucial role in presenting information to users.

Different applications require different display technologies depending on cost, power consumption, readability, brightness, and graphical capability. For example:

• LED displays are suitable for simple numeric indication.

• LCD displays are ideal for low-cost text-based output.

• OLED displays provide high-contrast graphical output.

However, each display type requires separate libraries and hardware configurations. This increases complexity during development and slows down hardware experimentation.

The objective of this project is to design and implement a universal firmware system that supports LED, LCD, and OLED displays using a single Arduino program. The goal is to create a reusable and scalable display framework that reduces development effort and enhances flexibility.

3. Motivation

During embedded system prototyping, developers frequently switch between different display modules depending on availability, cost constraints, power requirements, or specific project objectives. In real laboratory and academic environments, it is common to replace a display simply because another module is unavailable or a higher-resolution output is required.

However, in traditional workflows, switching displays requires:

• Changing libraries

• Rewriting initialization code

• Modifying communication settings

• Updating output functions

Each display technology follows a different communication protocol such as GPIO-based control (LED), I2C (LCD), or buffered graphical communication (OLED). As a result, the firmware structure becomes tightly coupled with the hardware. When hardware changes, software must also be modified accordingly. This tight coupling reduces portability and increases debugging complexity. Moreover, repeated rewriting of initialization routines and output functions increases the probability of logical errors, memory mismanagement, and compatibility issues. In resource-constrained microcontrollers like Arduino Uno, inefficient coding can also lead to SRAM overflow and unstable performance.

4. Literature Review

Existing display libraries such as LiquidCrystal (for LCD) and Adafruit SSD1306 (for OLED) are designed specifically for individual hardware modules. These libraries provide optimized functions for initialization, command transmission, and display updates tailored to particular controllers. While they are efficient and reliable for their intended hardware, they operate independently and do not offer a standardized interface that allows seamless switching between fundamentally different display technologies. Most traditional Arduino-based projects directly embed hardware-specific commands within the application code. This tightly coupled design reduces portability and increases dependency on specific hardware modules. When the display is changed, significant portions of the code must be modified, including initialization routines, data formatting logic, and communication configurations. Research in embedded systems highlights

the importance of Hardware Abstraction Layers (HAL) and modular driver architectures to improve portability, maintainability, and scalability. In large embedded platforms and real-time operating systems (RTOS), layered design is a common practice. The hardware layer handles low-level register operations, the driver layer manages device-specific protocols, and the application layer remains hardware-independent. This separation allows systems to scale efficiently and simplifies long-term maintenance.

However, such abstraction mechanisms are less commonly implemented in small-scale microcontroller platforms like Arduino Uno due to limited SRAM (2KB) and flash memory constraints. Developers often prioritize simplicity over architectural modularity to conserve memory. As a result, many small embedded projects lack structured abstraction, leading to firmware redundancy and reduced reusability.

5. Display Technology Overview

5.1 LED (7-Segment Display)

A 7-segment display consists of seven individual LED segments arranged to form digits from 0 to 9. Each segment is controlled independently through digital output pins. The display can be either common anode or common cathode type.

Advantages:

- Simple control logic
- Fast response time
- High brightness

Limitations:

- Supports only numeric characters
- Requires multiple GPIO pins

5.2 LCD (16x2 Character Display)

The 16x2 LCD module displays 16 characters per row across two rows. It operates using the HD44780 controller and can communicate via parallel interface or I2C.

Advantages:

- Low cost
- Low power consumption
- Easy text display

Limitations:

- Limited graphical capability
- Fixed character format

5.3 OLED (SSD1306 Display)

OLED displays generate light directly from organic compounds, eliminating the need for backlighting. The 0.96” SSD1306 OLED supports 128×64 pixel resolution.

Advantages:

- High contrast
- Wide viewing angle
- Supports graphics and text

Limitations:

- Higher memory requirement
- Slightly higher cost compared to LCD

6. System Architecture

The proposed system follows a well-structured layered architecture consisting of four clearly defined layers, each with a specific responsibility.

1. **Application Layer** – This is the topmost layer where the user or main program sends display messages. It focuses only on *what* needs to be displayed, without worrying about how the hardware actually works.
2. **Abstraction Layer** – This layer acts as a bridge between the application and the hardware drivers. It provides unified and common display functions so that the application can use a single interface, regardless of the display type being used.
3. **Driver Layer** – This layer contains device-specific implementations. Each display module (such as LCD, OLED, or 7-segment display) has its own driver code that handles its unique communication protocol and control logic.
4. **Hardware Interface Layer** – This is the lowest layer, responsible for managing hardware-level communication such as I2C, SPI, and GPIO operations. It directly interacts with the microcontroller pins and physical devices.

This modular and layered design ensures clear separation of concerns and loose coupling between hardware and software components. As a result, the system becomes more flexible, maintainable,

and scalable. If a new display type needs to be integrated in the future, developers only need to implement a new driver module without modifying the application layer, making the overall system more efficient and easy to extend.

7. Components & Hardware Overview

7.1 Component List

The following components are used in this project:

Component	Model / Type	Purpose
Arduino Board	Arduino Uno	Main controller
7-Segment Display	Common Cathode	Numeric output
LCD Module	16x2 with I2C	Text output
OLED Module	0.96” SSD1306	Graphical output
Breadboard	400-point	Circuit assembly
Resistors	220Ω	Current limiting
Jumper Wires	Male–Male	Interconnection

7.2 Circuit Connections

Each display module is wired to the Arduino as described below. All connections use the 5V and GND rails on the breadboard.

Display / Component	Arduino Pin(s)	Connections	Working Principle
Arduino Board	—	Provides 5V and GND to all display modules	The microcontroller controls and coordinates all connected display devices, sending appropriate signals based on program logic.
OLED Display (SSD1306)	A4 and A5	VCC → 5V GND → GND SDA → A4 SCL → A5	The OLED communicates with the Arduino using the I2C protocol through SDA (A4) and SCL (A5). It receives graphical data and renders text or images on the screen.
LCD Display (16x2 I2C)	A4 and A5	VCC → 5V GND → GND SDA → A4 SCL → A5	The LCD module uses I2C communication to receive character data from the Arduino. It displays alphanumeric text based on commands sent by the controller.
7-Segment Display (Common Cathode)	D2 – D8	Segment a → D2 (via 220Ω) Segment b → D3 (via 220Ω) Segment c → D4 (via 220Ω) Segment d → D5 (via 220Ω) Segment e → D6 (via 220Ω) Segment f → D7 (via 220Ω) Segment g → D8 (via 220Ω) Common Cathode → GND	Each segment is controlled individually by digital output pins. When a pin is set HIGH, current flows through the resistor and lights the corresponding segment, forming numeric digits.

8. Software Implementation

8.1 Program Architecture

The firmware is developed in Arduino IDE using structured programming principles. During the setup phase, I2C communication and GPIO pins are initialized. The system attempts to detect OLED and LCD modules using predefined I2C addresses.

If no I2C display is detected, the firmware defaults to 7-segment control mode.

A unified function such as displayMessage(String msg) ensures that the same command works for all display types.

8.2 Memory Considerations

Arduino Uno contains 2KB SRAM. OLED graphics buffer requires approximately 1KB memory. Efficient memory management is implemented to prevent overflow. Only one display buffer is active at a time.

8.3 Source Code

```
#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#include <Adafruit_GFX.h>

#include <Adafruit_SSD1306.h>

/* OLED Display Configuration */

#define SCREEN_WIDTH 128

#define SCREEN_HEIGHT 64


Adafruit_SSD1306 oled(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

LiquidCrystal_I2C lcd(0x27, 16, 2);


/* Device Detection Flags */

bool oledFound = false;

bool lcdFound  = false;


/* 7-Segment Display Pin Configuration */

int segPins[7] = {2, 3, 4, 5, 6, 7, 8};


/* Digit Patterns (0–9) for Common Cathode 7-Segment */

byte digits[10][7] =

{

  {1,1,1,1,1,1,0}, // 0

  {0,1,1,0,0,0,0}, // 1

  {1,1,0,1,1,0,1}, // 2

  {1,1,1,1,0,0,1}, // 3
```

```
  {0,1,1,0,0,1,1}, // 4
  {1,0,1,1,0,1,1}, // 5
  {1,0,1,1,1,1,1}, // 6
  {1,1,1,0,0,0,0}, // 7
  {1,1,1,1,1,1,1}, // 8
  {1,1,1,1,0,1,1}  // 9
};

/* Function to Check I2C Devices */
void checkI2C()
{
  Wire.beginTransmission(0x3C);   // OLED address
  if (Wire.endTransmission() == 0)
    oledFound = true;

  Wire.beginTransmission(0x27);   // LCD address
  if (Wire.endTransmission() == 0)
    lcdFound = true;
}

/* Function to Display Number on 7-Segment */
void displaySevenSegment(int num)
{
  for (int i = 0; i < 7; i++)
  {
    digitalWrite(segPins[i], digits[num][i]);
  }
}
```

```
/* Setup Function */
void setup()
{
  Wire.begin();
  checkI2C();
  /* If OLED is Detected */
  if (oledFound)
  {
    oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
    oled.clearDisplay();
    oled.setTextSize(2);
    oled.setTextColor(SSD1306_WHITE);
    oled.setCursor(0, 20);
    oled.println("OLED Mode");
    oled.display();
  }

  /* If LCD is Detected */
  else if (lcdFound)
  {
    lcd.init();
    lcd.backlight();
    lcd.setCursor(0, 0);
    lcd.print("LCD Mode");
    lcd.setCursor(0, 1);
    lcd.print("Universal");
  }
```

```
  /* If No I2C Display Found → Use 7-Segment */
  else
  {
    for (int i = 0; i < 7; i++)
    {
      pinMode(segPins[i], OUTPUT);
    }

    displaySevenSegment(8);   // Display digit 8
  }
}

/* Main Loop */
void loop()
{
  // No repeated operation required
}
```

9. Working Principle

Step 1: Initialization

When the system powers on, the Arduino initializes the required communication protocols such as I2C and configures the necessary pins. It prepares the display modules for operation and performs basic setup routines to ensure that all connected devices are ready for communication.

Step 2: Display Selection

After initialization, the firmware checks for available display modules in a predefined order. It first attempts to detect the OLED display using its I2C address. If the OLED is not found, it then checks for the LCD module. If neither I2C display responds, the system automatically switches to the 7-segment LED display mode. This automatic detection ensures smooth operation without manual configuration.

Step 3: Message Output

Once a display is selected, the same message is sent through the appropriate driver logic for that specific device. The abstraction layer ensures that the application code does not need to change for different display types. Each driver handles the hardware-specific implementation while maintaining a common interface.

This structured approach improves flexibility, enhances system reliability, and simplifies future maintenance or upgrades.

10. Experimental Results

The system was tested individually with all three display modules to verify proper functionality and compatibility. Each display was connected separately to ensure accurate evaluation of performance and response.

Observations:

- The **OLED display** provided the clearest and sharpest output, with a noticeably faster refresh rate. Graphical content and text were displayed smoothly and with good visibility.
- The **LCD module** produced stable and consistent text output. It operated reliably with relatively low power consumption, making it suitable for simple text-based applications.
- The **7-segment display** accurately displayed numeric data as expected. It functioned correctly for basic number representation and demonstrated simple, reliable performance.

During testing, the firmware operated successfully without requiring any modifications when switching between different display hardware. This confirms that the abstraction-based design ensures hardware flexibility and seamless integration.

11. Output

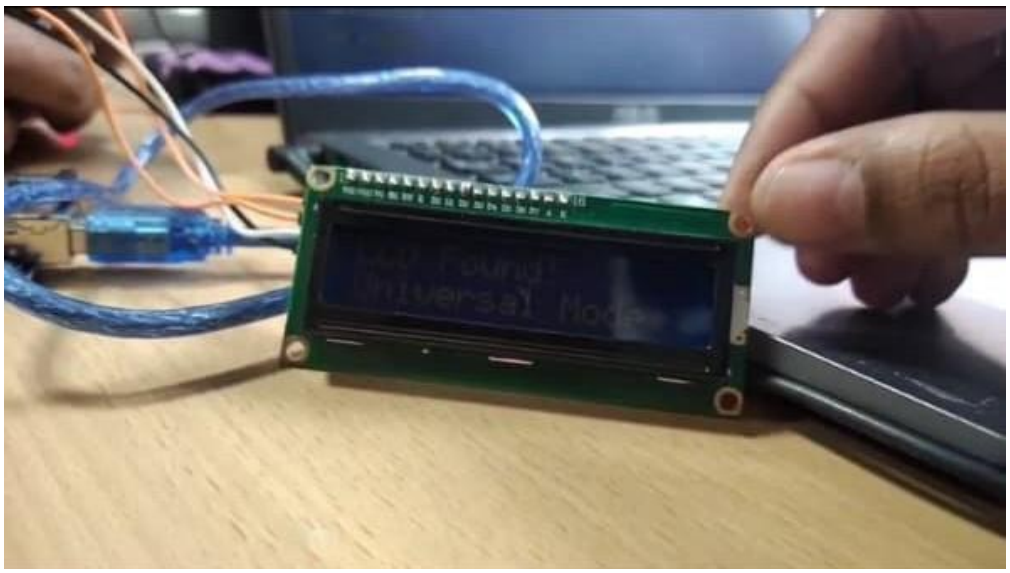

Fig2: Hardware Implementation – LCD Output

Fig3: Hardware Implementation – OLED Output

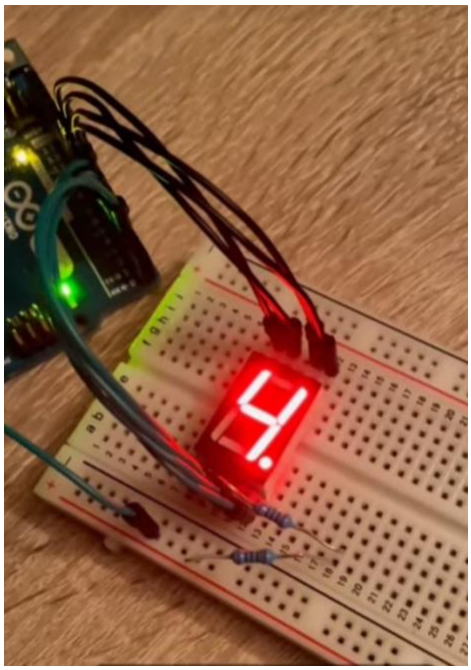

Fig4: Hardware Implementation – 7-Segment Output (Insert Image)

12. Conclusion

This project successfully demonstrates the implementation of a **Universal Display Driver** capable of supporting LED, LCD, and OLED displays using a single Arduino firmware architecture. By incorporating modular abstraction and a structured firmware design, the system removes the need to modify application-level logic when switching between different display hardware.

The layered approach clearly separates hardware control from application functionality, making the system more organized, maintainable, and scalable. It highlights that even resource-constrained microcontrollers, such as Arduino-based systems, can effectively implement reusable and extensible driver frameworks when designed properly.

Overall, this project serves as a practical and educational model for students, researchers, and embedded system developers who aim to build flexible prototyping platforms with minimal hardware dependency and maximum code reusability.